WHO MOVED MY HEAP · 03 DEFINITIONS

Stream API

ReferencePipeline, eager and lazy operations

05

What is a stream?

A sequence of elements processed by a pipeline: a source, intermediate operations, one terminal operation.

- Intermediate operations (`filter`, `map`) are **lazy**: they only record work.
- The terminal operation (`anyMatch`, `count`, `findFirst`) is **eager**: it runs the pipeline.
- Short-circuiting terminals stop at the first match.
- A stream can be used only once.

A pipeline in code

```
boolean found = list.stream()                     // source
                    .filter(each -> each > 500)   // intermediate (lazy)
                    .findFirst()                  // terminal (eager)
                    .isPresent();
```

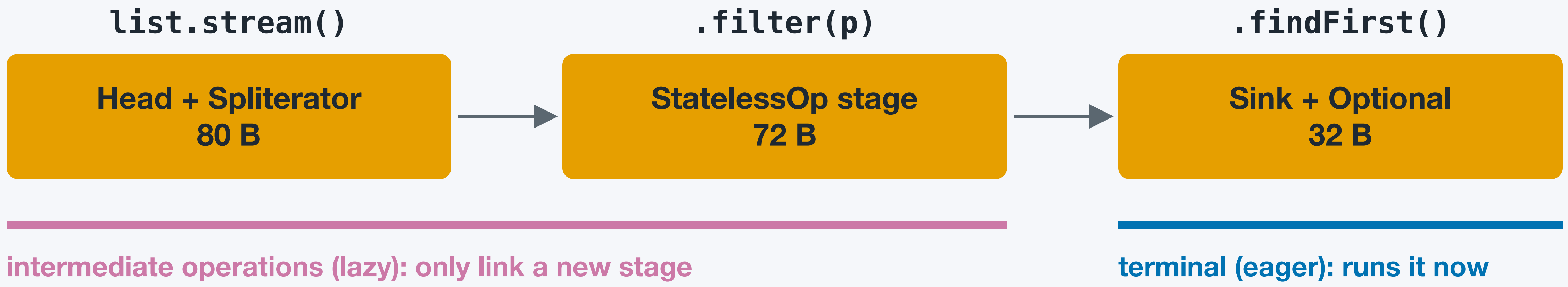
- Every call builds a new pipeline of objects.
- Nothing runs until `findFirst()`.

Inside: `ReferencePipeline`

The internal class behind `Stream<T>`. Primitive versions: `IntPipeline`, `LongPipeline`, `DoublePipeline`.

- `stream()` creates a `Head` stage and a `Spliterator`.
- Each intermediate operation links one new stage (`StatelessOp` or `StatefulOp`).
- The terminal operation creates a `TerminalOp`, chains `Sink` objects and pushes elements through them.

What one call allocates



list.stream().filter(p).findFirst(): what one call allocates

`list.stream()` → `.filter(p)` → `.findFirst()`

Head + Spliterator
80 B

StatelessOp stage
72 B

Sink + Optional
32 B

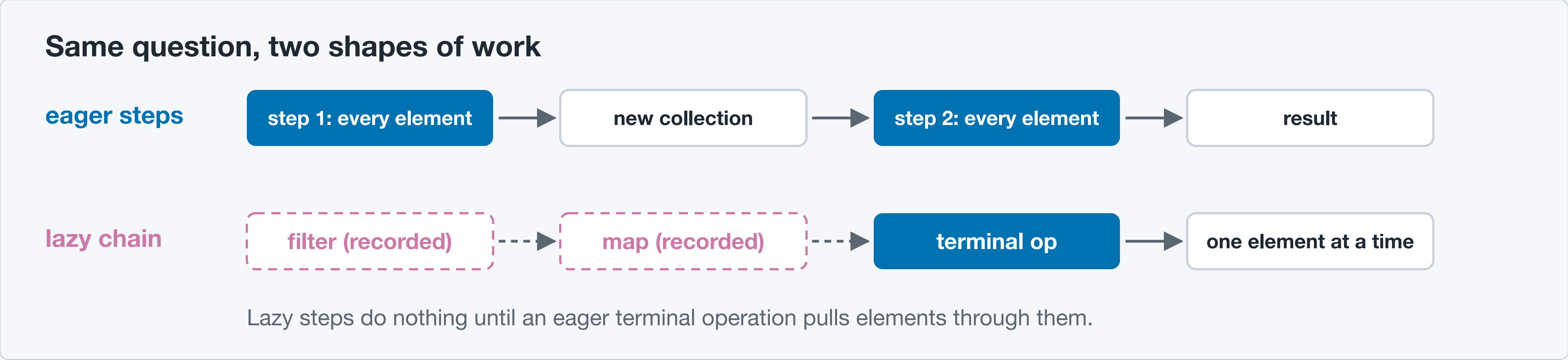
intermediate operations (lazy): only link a new stage

terminal (eager): runs it now

+ captured lambda 16 B = 200 B per call. The EC loop allocates 0 B.

- Escape analysis removes none of it.

Eager vs lazy



Same question, two shapes of work

eager steps

step 1: every element → new collection → step 2: every element → result

lazy chain

filter (recorded) ⇢ map (recorded) ⇢ terminal op → one element at a time

Lazy steps do nothing until an eager terminal operation pulls elements through them.

- **Eager**: runs when called and returns a result.
- **Lazy**: recorded; runs one element at a time when a result is needed.

Eager vs lazy in the JDK

	Eager	Lazy
When work happens	At each call	At the terminal operation
In-between results	A new collection per step	None
Short-circuit	Per step	Across the whole chain
JDK examples	`anyMatch`, `count`, `findFirst`, `collect`, `removeIf`, `forEach`	`filter`, `map`, `sorted`

References

- `java.util.stream` package summary (Java SE 25): pipelines, laziness, low-level construction
- `Stream` (Java SE 25 API)
- The Java Tutorials: Aggregate Operations
- The Java Tutorials: Reduction